

STA326: Data Science Practice

Wei Hongxin
2026 Spring

Quiz Time



* 1. Tanh 函数相比 Sigmoid 函数的优势是什么? [5分]

- ☒ 输出范围更大, 梯度最大值更高 (正确答案)
- ☐ 计算更简单, 训练更快
- ☐ 不会导致梯度消失
- ☐ 在负值时保留更多信息

* 2. 在深度网络中, 梯度爆炸的表现可能是: [5分]

- ☐ 前层权重更新过慢
- ☒ 损失函数值突然变成 NaN (正确答案)
- ☐ 隐藏层输出始终为 0
- ☐ 训练收敛速度加快

* 3. Hinton 的 AlexNet 使用了哪种激活函数来加速训练?

☐ Sigmoid

☐ Tanh

☒ ReLU

☐ Softmax



* 4. 在单隐藏层 MLP 中，如果不使用激活函数，增加隐藏单元数量会怎样? [5分]

☐ 提高模型的非线性表达能力

☒ 不会改变模型的表达能力 (正确答案)

☐ 导致梯度爆炸

☐ 使模型能近似任意函数

* 5. Sigmoid 函数的梯度最大值是多少? [5分]

☐ 1

☐ 0.5

☒ 0.25 (正确答案)

☐ 0

* 6. 下列哪种激活函数的输出范围是 $(-1, 1)$? [5分]

☐ ReLU

☐ Sigmoid

☒ Tanh (正确答案)

☐ Softmax

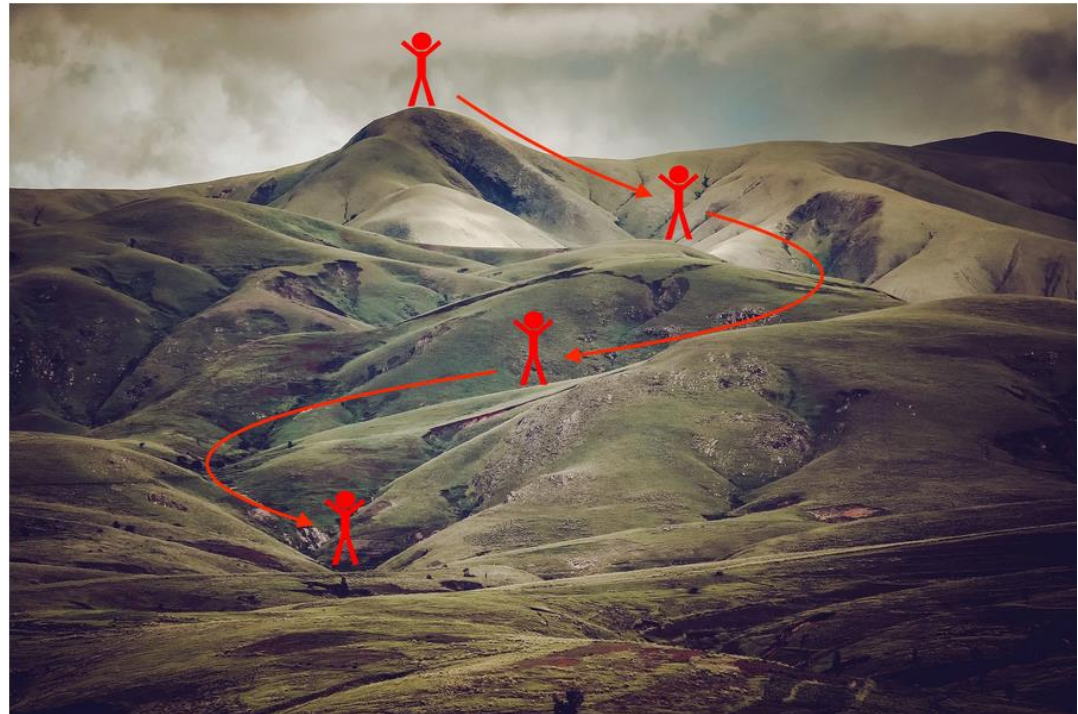
* 7. 下列哪种情况会导致梯度消失问题?

[5分]

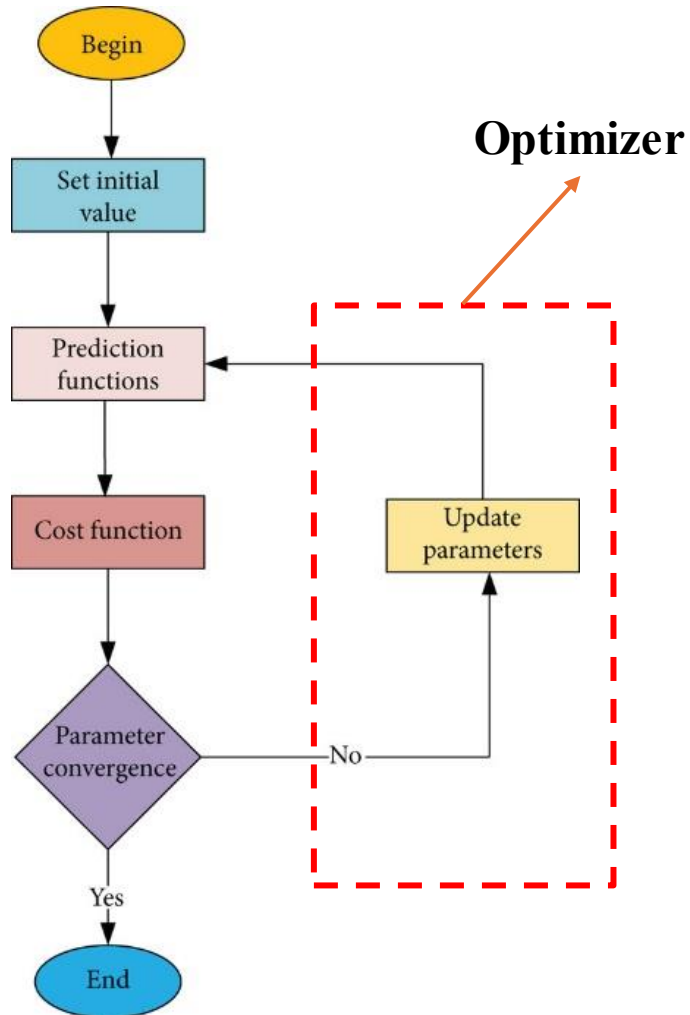
- ☐ 使用 ReLU 激活函数
- ☒ 使用 Sigmoid 激活函数, 且输入值很大 (正确答案)
- ☐ 权重初始化为随机值
- ☐ 使用梯度裁剪

Outlines

- Optimization Challenges in Deep Learning
- Minibatch SGD
- Advanced optimizers



The role of optimization



$$\theta^* = \arg \min_{\theta} \sum_{(x,y) \in D} \mathcal{L}(f_{\theta}(x), y)$$

The equation is annotated with colored circles and arrows: a green circle around $\arg \min_{\theta}$ points to 'gradient descent'; a blue circle around $\sum_{(x,y) \in D}$ points to 'dataset'; a red circle around $\mathcal{L}$ points to 'loss'; and an orange circle around f_{θ} points to 'neural network'.

What is expected to optimized?

The Expected Risk (Generalization error):

$$R(f) = E_Z[L(z, f)] = \int_Z L(z, f)p(z)dz$$

The expected loss on the entire population of data

But we do not know $p(z)$, and we cannot test all possible z .

What is optimized actually?

The Empirical Risk (an average loss on the training dataset):

$$\hat{R}(f, D_n) = \frac{1}{n} \sum_{i=1}^n L(z_i, f)$$

D_n is the training dataset.

Does it always achieve the expectation?

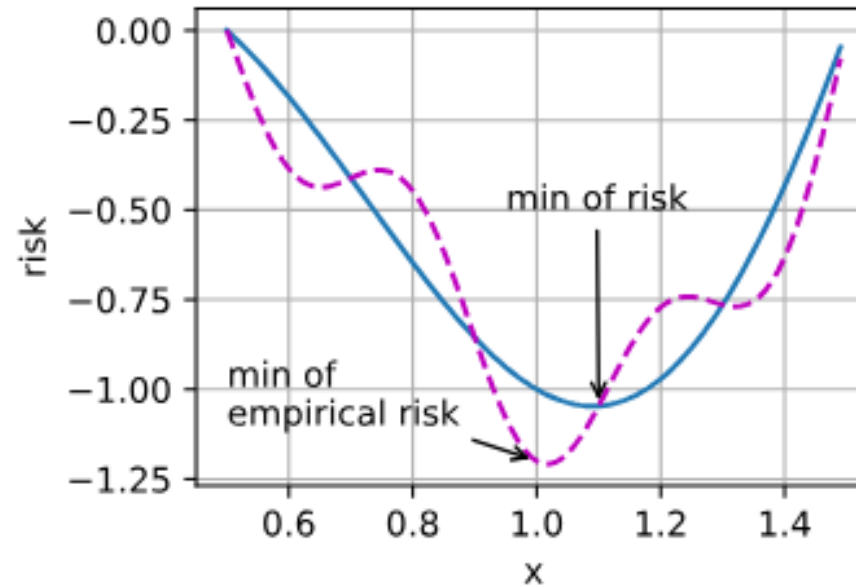
Does this make sense?

The empirical risk is an unbiased estimate of the risk under **iid assumption**.

$$E \left(\hat{R}(f, D) \right) = R(f)$$

$$\begin{aligned} E[\hat{R}(f, D)] &= E \left[\frac{1}{n} \sum_{i=1}^n L(f, Z_i) \right] \\ &= \frac{1}{n} \sum_{i=1}^n E[L(f, Z_i)], && Z_i \text{ s are independent} \\ &= \frac{1}{n} \sum_{i=1}^n E[L(f, Z)], && Z_i \text{ s are identically distributed} \\ &= \frac{1}{n} n E[L(f, Z)] \\ &= R(f) \end{aligned}$$

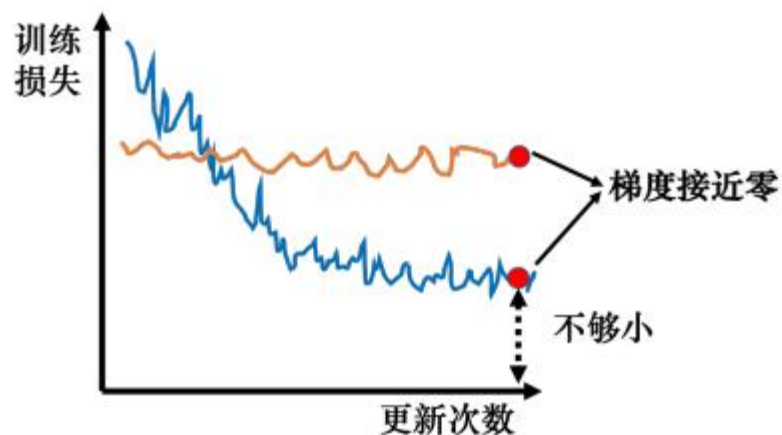
Difference between the two risks



Think: Why do we prefer a flat minima of empirical risk?

When will we stop?

The optimization will converge at zero gradients (Critical Points).



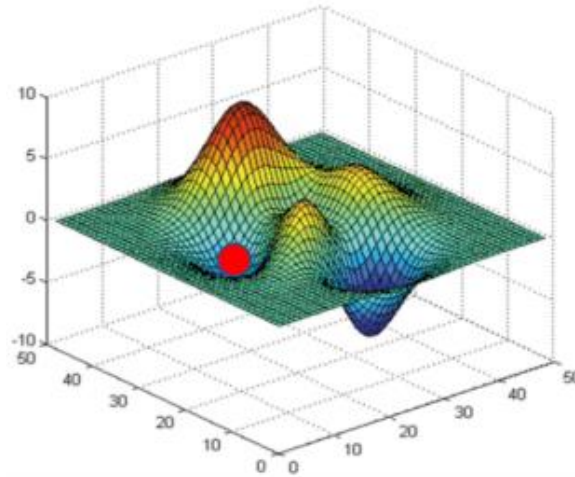
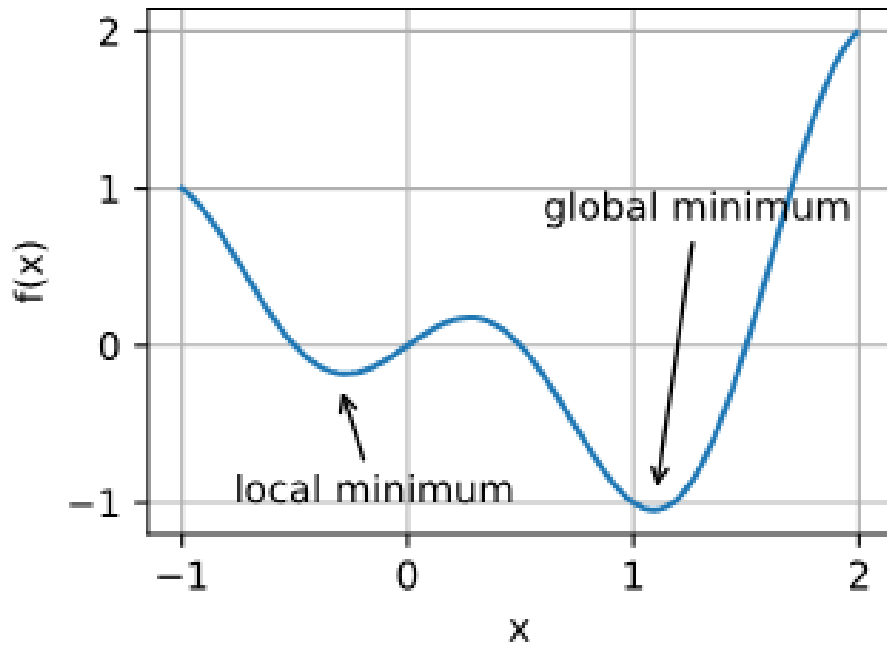
Case 1: **Global** minimum

Case 2: **Local** minimum

Case 3: Saddle point

Local Minima

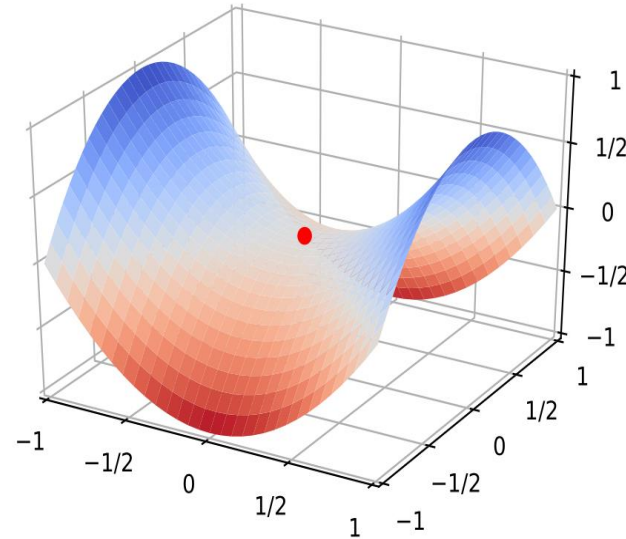
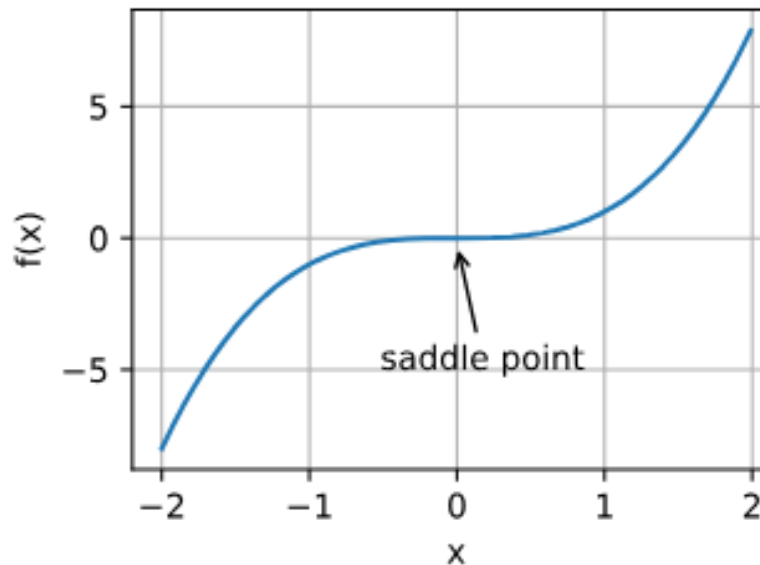
A point where the loss function reaches a minimum within a small region, but not the global minimum.



- Gradient is near zero ($\nabla f \approx 0$).
- Optimization may get "stuck," unable to reach the global optimum.

Saddle Points

A point where the loss function decreases in some directions and increases in others, resembling a saddle.

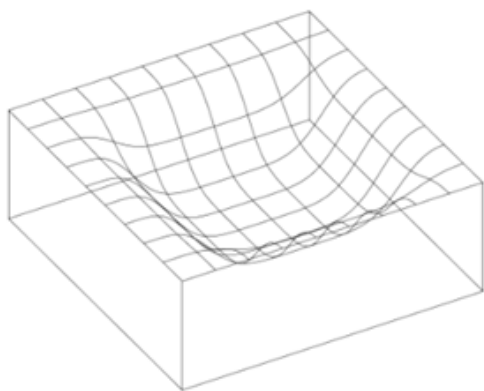


- Gradient is near zero ($\nabla f \approx 0$), but not an extremum.
- Hessian matrix has both **positive** and **negative** eigenvalues.

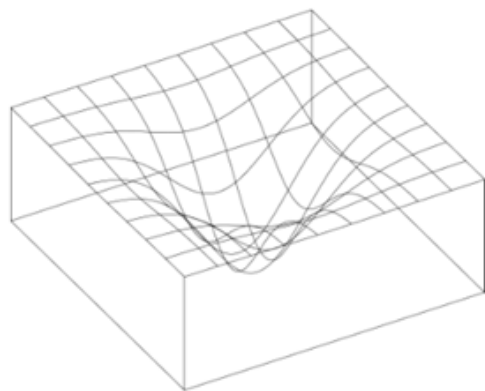
Common in **high-dimensional spaces** (e.g., deep learning), making optimization more challenging.

Flat Minima

一个平坦最小值的邻域内，所有点对应的训练损失都比较接近



(a) 平坦最小值



(b) 尖锐最小值

- 大部分的局部最小解是等价的
- 局部最小解对应的训练损失都可能非常接近于全局最小解对应的训练损失

Why Flat minima?

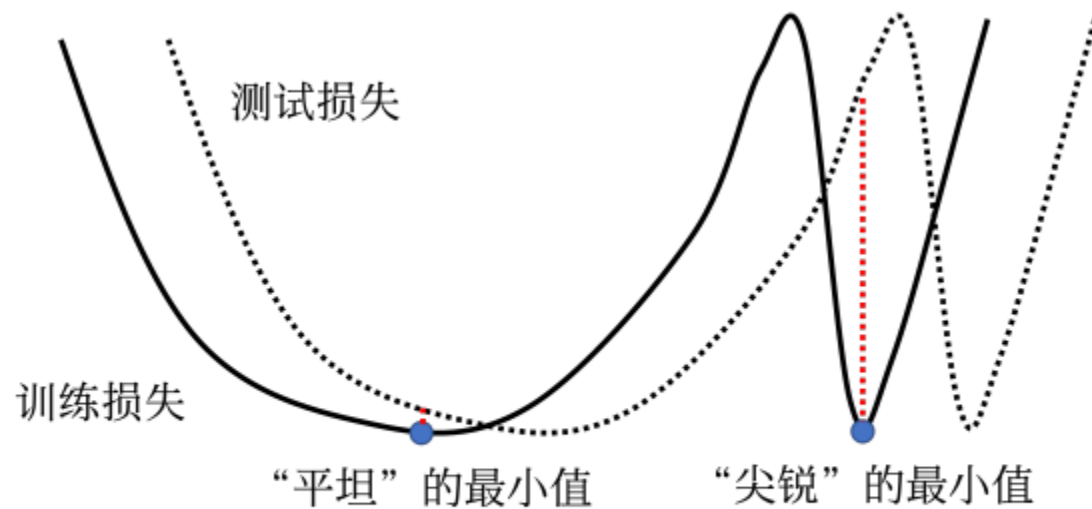
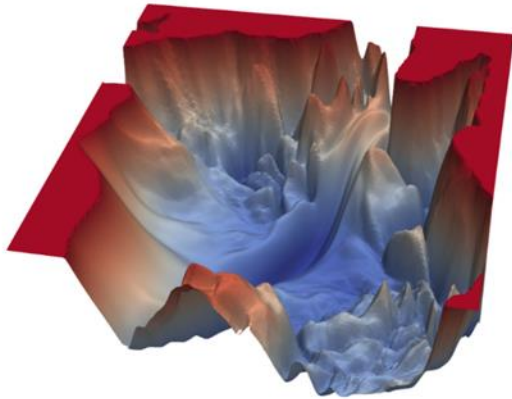


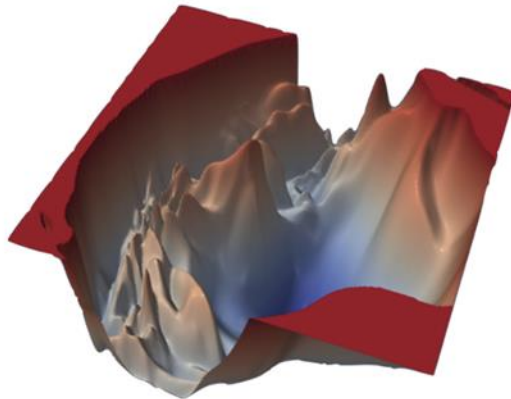
图 3.13 小批量优化容易跳出局部最小值的原因

VISUALIZING THE LOSS LANDSCAPE

VGG-56

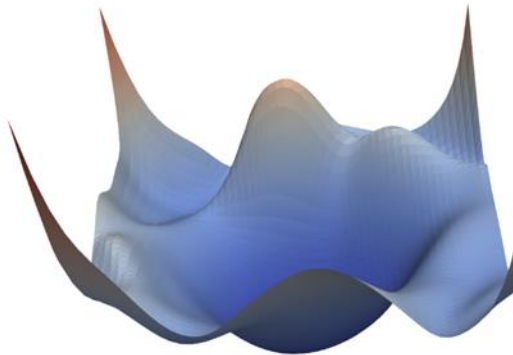


VGG-110

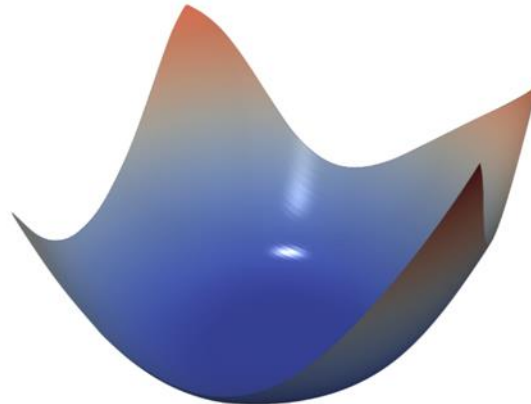


without skip connections

Resnet-56



Densenet-121



with skip connections

Local Minima vs. Saddle Points

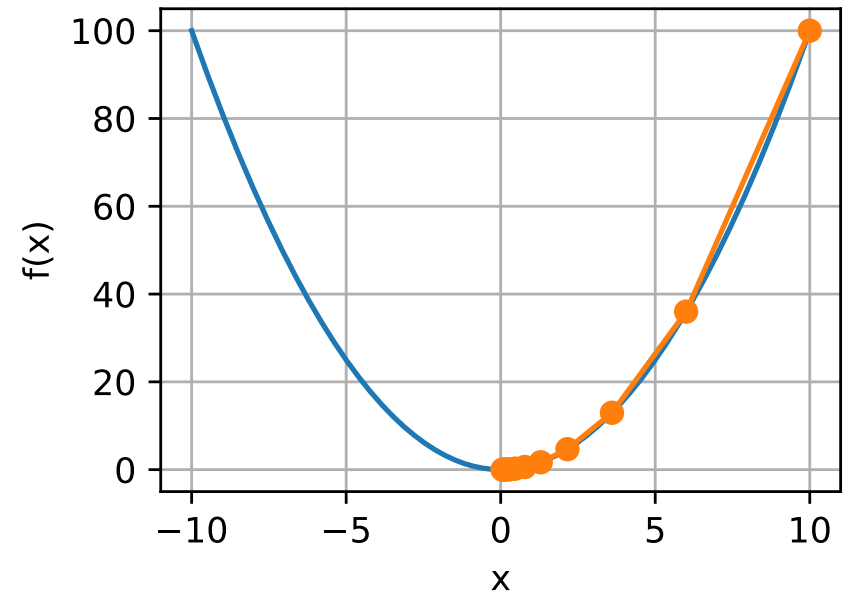
Feature	Local Minima	Saddle Points
Gradient	≈ 0	≈ 0
Surroundings	Rises all around	Falls in some directions
Optimization Impact	May halt progress	May slow convergence

Gradient Decent

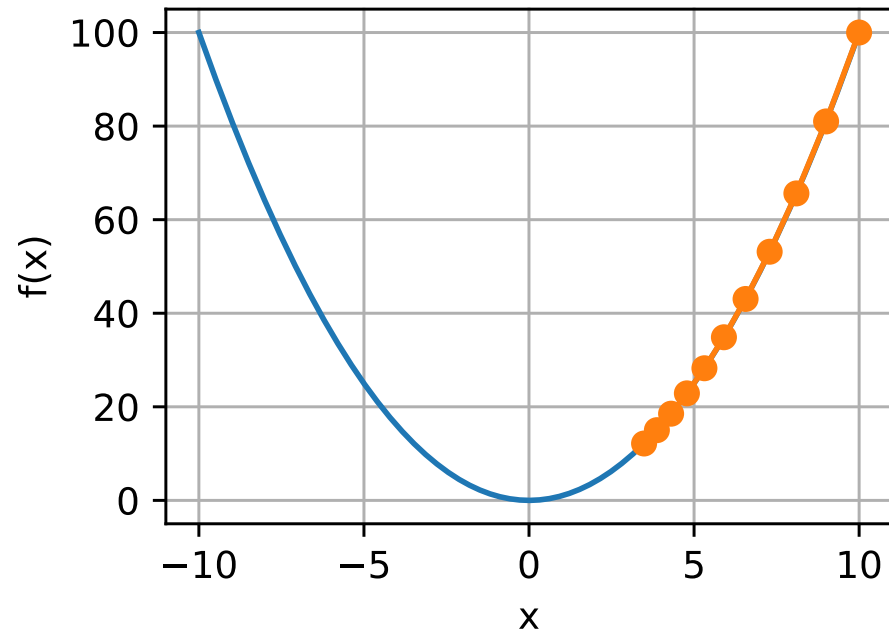
An iterative algorithm to **minimize** a function by moving in the direction **opposite** to the gradient.

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t)$$

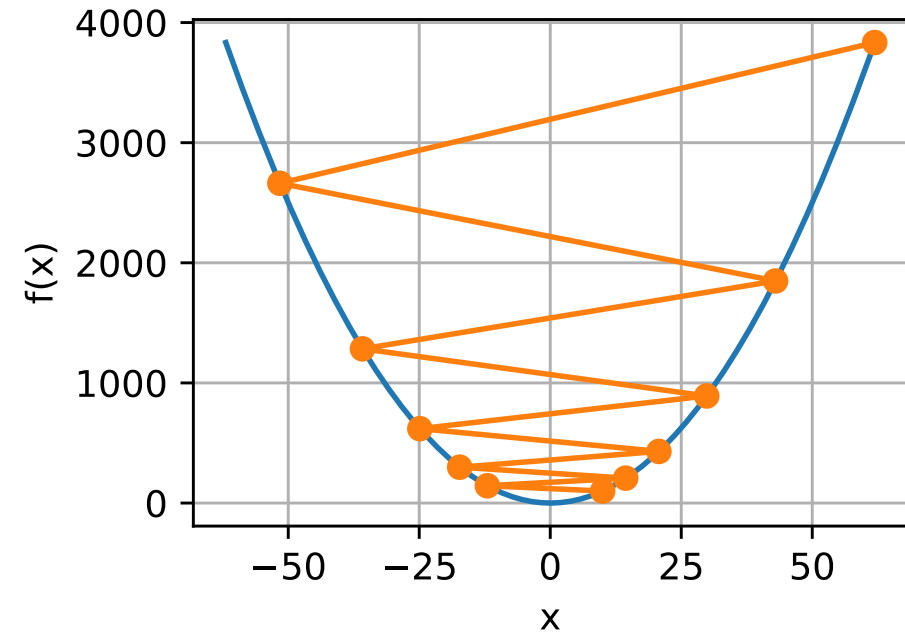
- θ : Parameters
- η : Learning rate
- $\nabla L(\theta_t)$: Gradient of the loss function L .



The effect of learning rate

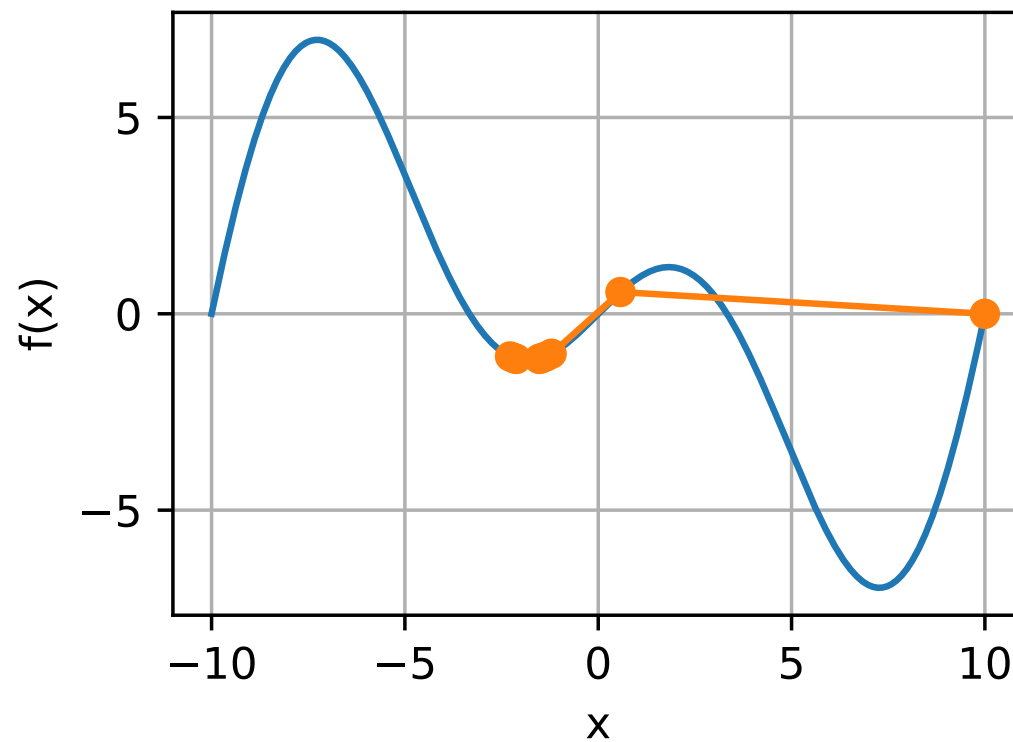


Small LR



Large LR

LR in local minima



Pros and Cons of GD

Advantages:

1. Converges to the **global** minimum for **convex** functions.
2. Simple to implement.

Disadvantages:

1. **High** computational cost: **Uses the entire dataset per iteration.**
2. Slow for large datasets.
3. May get stuck in **local minima** for non-convex functions.

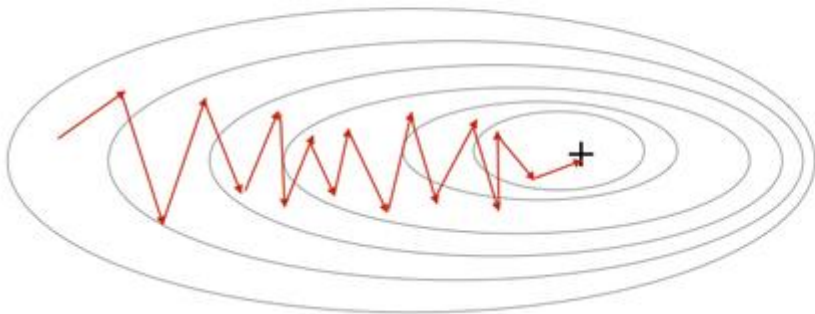
What is SGD?

Stochastic Gradient Descent (SGD)

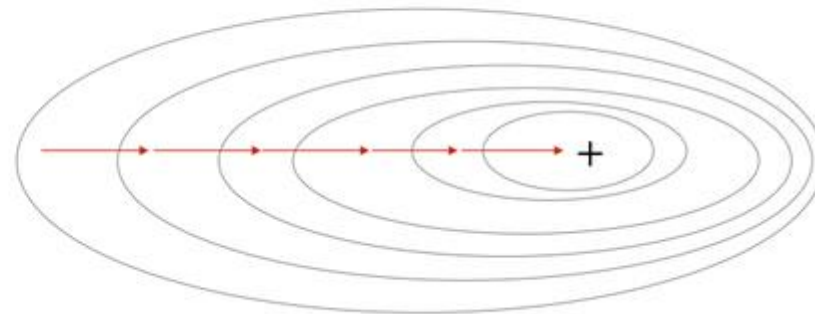
A variant of GD that updates parameters using the gradient of a **single random** sample.

$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t; x_i, y_i)$$

Stochastic Gradient Descent



Gradient Descent



随机梯度下降算法

算法 2.1 随机梯度下降法

输入: 训练集 $\mathcal{D} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 验证集 $\mathcal{V}$, 学习率 α

1 随机初始化 θ ;

2 **repeat**

3 对训练集 $\mathcal{D}$ 中的样本随机排序;

4 **for** $n = 1 \cdots N$ **do**

5 从训练集 $\mathcal{D}$ 中选取样本 $(\mathbf{x}^{(n)}, y^{(n)})$;

6 $\theta \leftarrow \theta - \alpha \frac{\partial \mathcal{L}(\theta; \mathbf{x}^{(n)}, y^{(n)})}{\partial \theta}$; // 更新参数

7 **end**

8 **until** 模型 $f(\mathbf{x}; \theta)$ 在验证集 $\mathcal{V}$ 上的错误率不再下降;

输出: θ

SGD: Advantages and Disadvantages

Pros:

1. **High** efficiency: Only one sample per iteration.
2. Fluctuations can help **escape local minima**.

Cons:

1. **Unstable** updates, leading to slower final convergence.
2. May not reach the exact minimum.

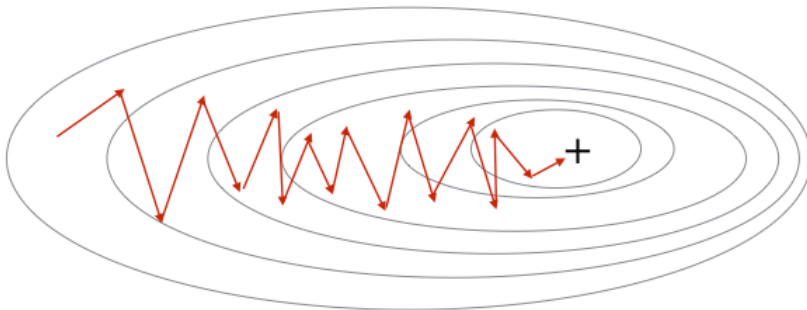
What is Mini-batch SGD

A compromise between GD and SGD, using a **small batch** of samples per iteration.

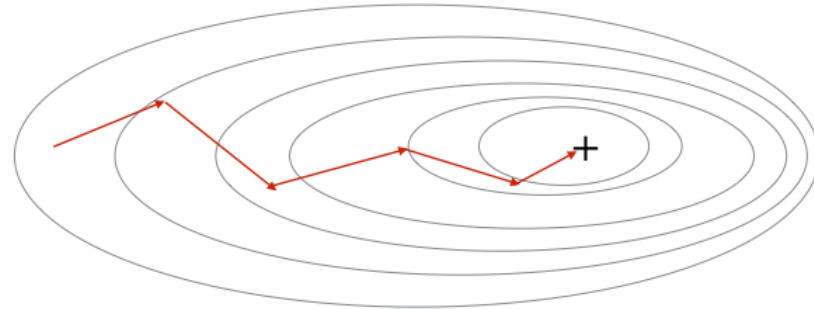
$$\theta_{t+1} = \theta_t - \eta \nabla L(\theta_t; X_b, Y_b)$$

(X_b, Y_b) : A mini-batch of samples.

Stochastic Gradient Descent



Mini-Batch Gradient Descent



Mini-batch SGD: Pros and Cons

Pros:

1. Balances efficiency and stability.
2. Leverages matrix operations (e.g., on GPUs) for speed.
3. Ideal for deep learning and large datasets.

Cons:

1. Requires tuning of batch size (e.g., 32, 64, 256).
2. Still vulnerable to local minima.

GD vs. SGD vs. Mini-batch SGD

Aspect	GD	SGD	Mini-batch SGD
Efficiency	Low (full data)	High (1 sample)	Medium (batch)
Convergence	Stable but slow	Fast but noisy	Balanced
Use Case	Small datasets	Large datasets	Deep learning

Effect of Batch size

- **Larger** batch size, **Smaller** Variance of gradient, **Smaller** Noise
 - can use **large** learning rate.
- **Smaller** batch size, **Larger** Variance of gradient, **Larger** Noise
 - Should use **smaller** learning rate.

How to improve SGD?

- Learning rate

- Learning rate decay: (scheduler)
- Adagrad
- Adadelata
- RMSprop

$$\Delta\theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t$$

$$\Delta\theta_t = -\alpha \mathbf{g}_t$$

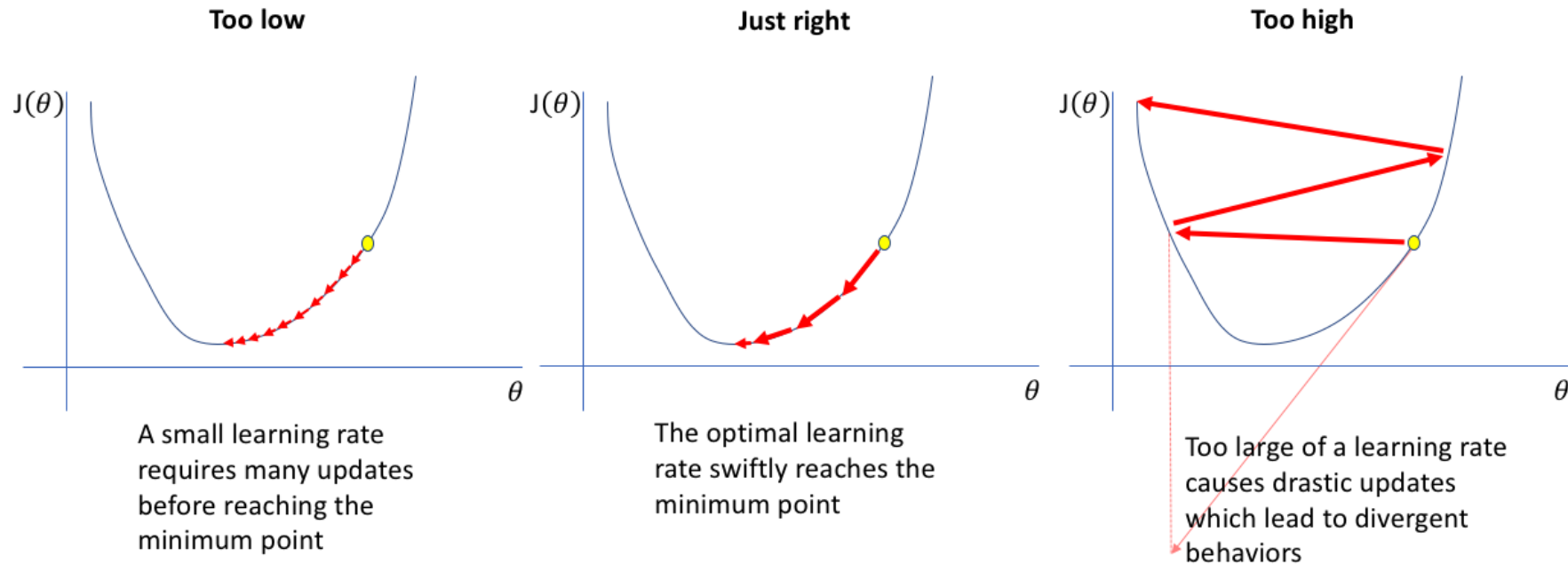
Adam

- Gradient

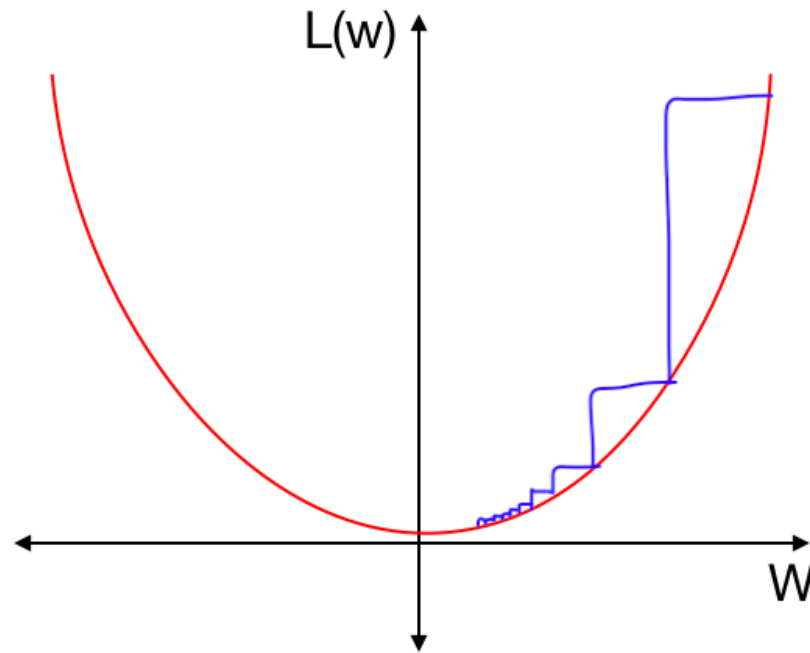
- Momentum
- Nesterov accelerated gradient
- Gradient clipping

$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t$$

Effect of learning rate

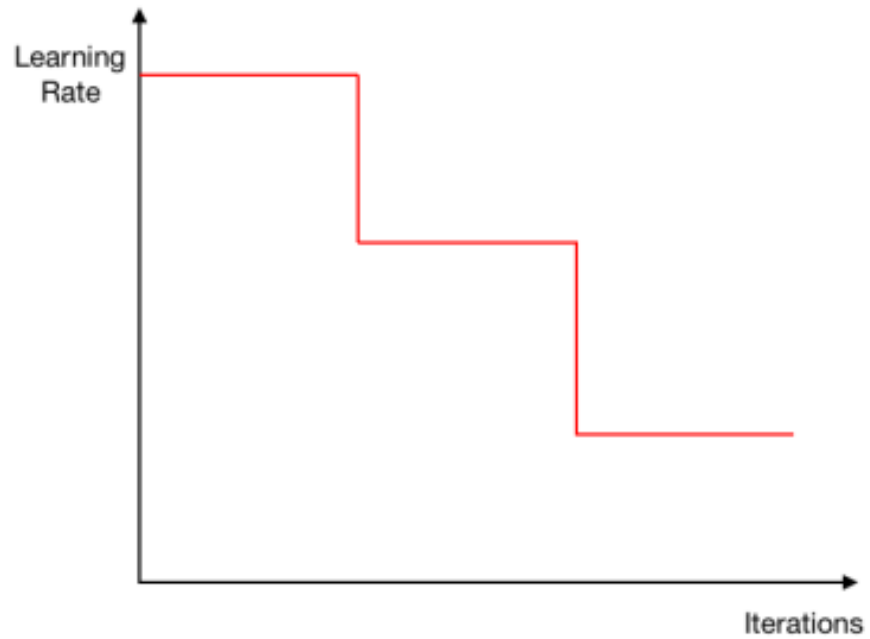


Learning rate Decay

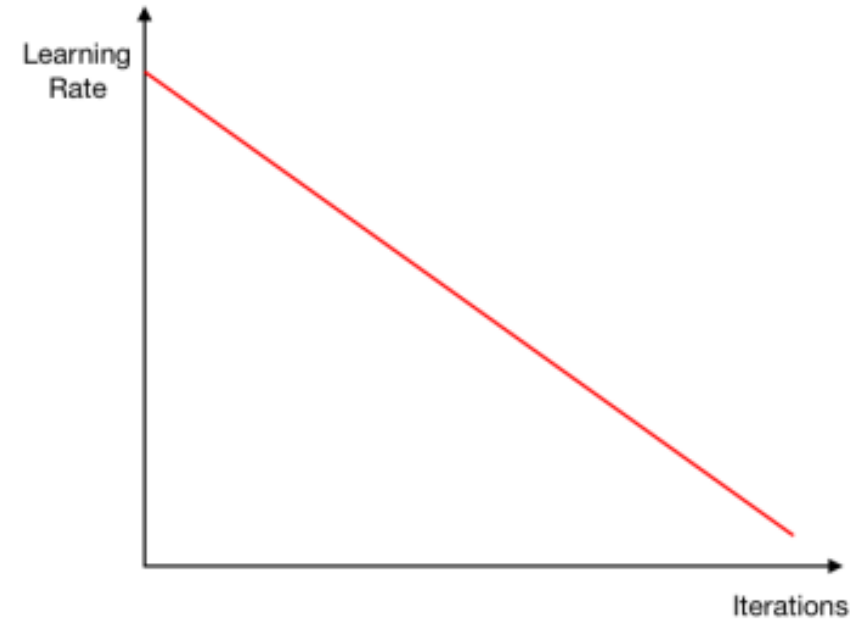


<https://pytorch.org/docs/stable/optim.html#module-torch.optim>

Learning rate decay

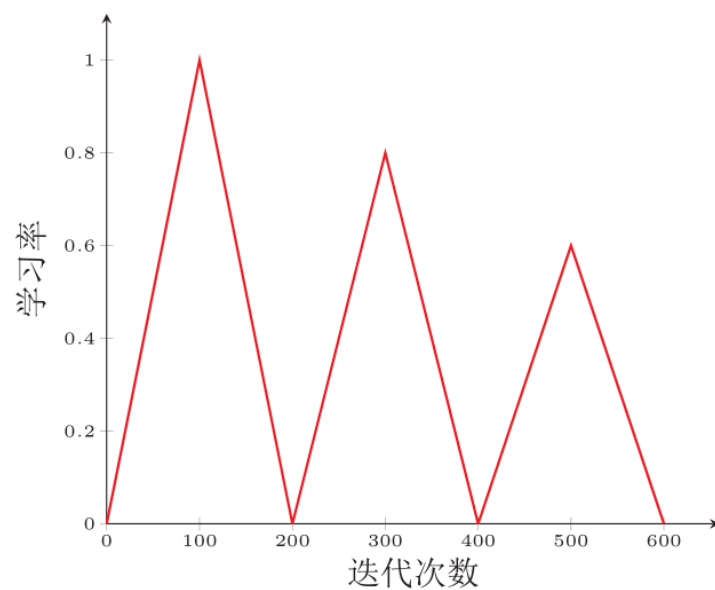


Step Decay

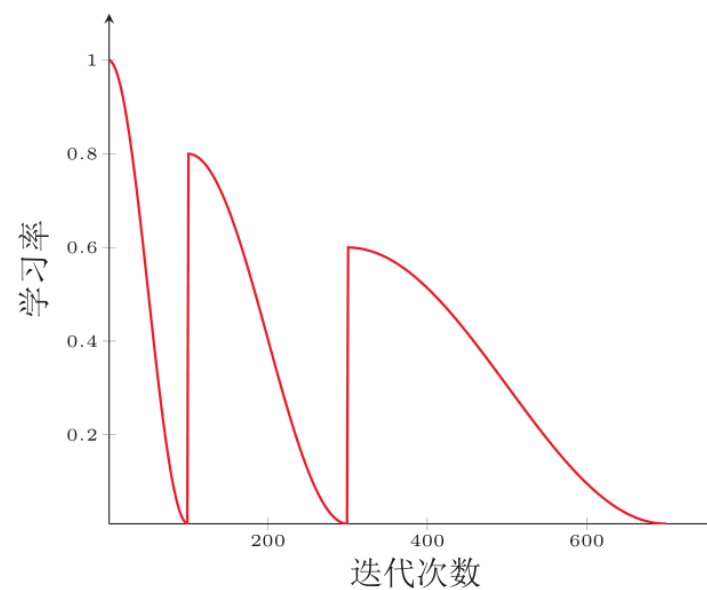


Linear Decay

Cyclical Learning Rates

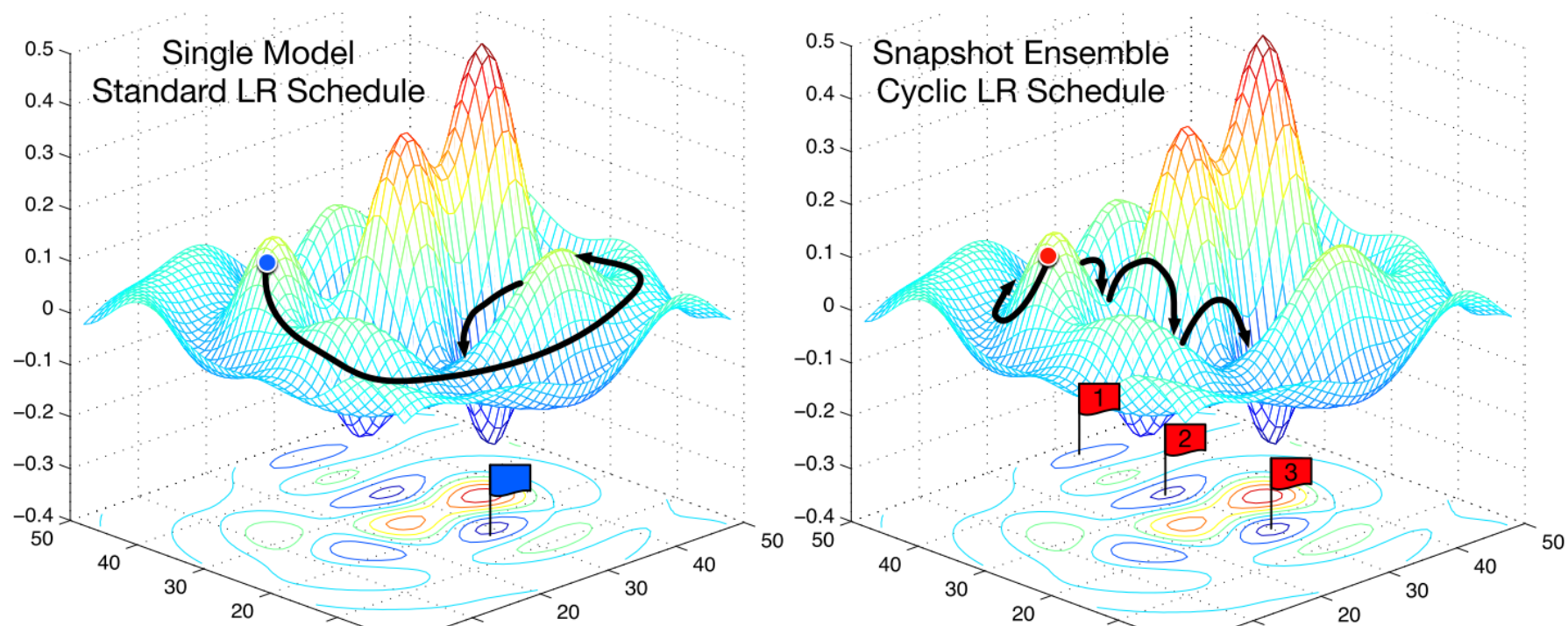


(a) 三角循环学习率



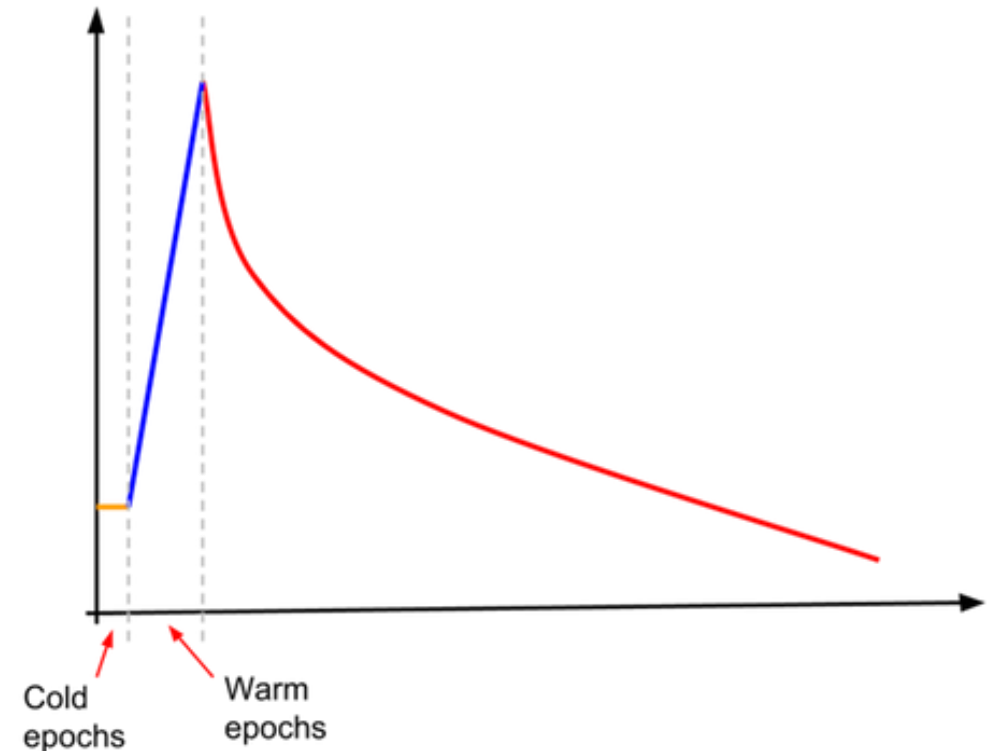
(b) 带热重启的余弦衰减

Cyclical Learning Rates



Others

- Don't Decay the Learning Rate, Increase the Batch Size
 - <https://openreview.net/pdf?id=B1Yy1BxCZ>
- Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour
 - Warmup
 - <https://arxiv.org/abs/1706.02677>



Adaptive learning rate

Adagrad

RMSprop

Adadelta

$$\Delta\theta_t = -\frac{\alpha}{\sqrt{G_t + \epsilon}} \odot \mathbf{g}_t$$

$$G_t = \sum_{\tau=1}^t \mathbf{g}_{\tau} \odot \mathbf{g}_{\tau}$$

$$\begin{aligned} G_t &= \beta G_{t-1} + (1 - \beta) \mathbf{g}_t \odot \mathbf{g}_t \\ &= (1 - \beta) \sum_{\tau=1}^t \beta^{t-\tau} \mathbf{g}_{\tau} \odot \mathbf{g}_{\tau} \end{aligned}$$

$$\Delta\theta_t = -\frac{\sqrt{\Delta X_{t-1}^2 + \epsilon}}{\sqrt{G_t + \epsilon}} \mathbf{g}_t$$

$$\Delta X_{t-1}^2 = \beta_1 \Delta X_{t-2}^2 + (1 - \beta_1) \Delta\theta_{t-1} \odot \Delta\theta_{t-1}$$

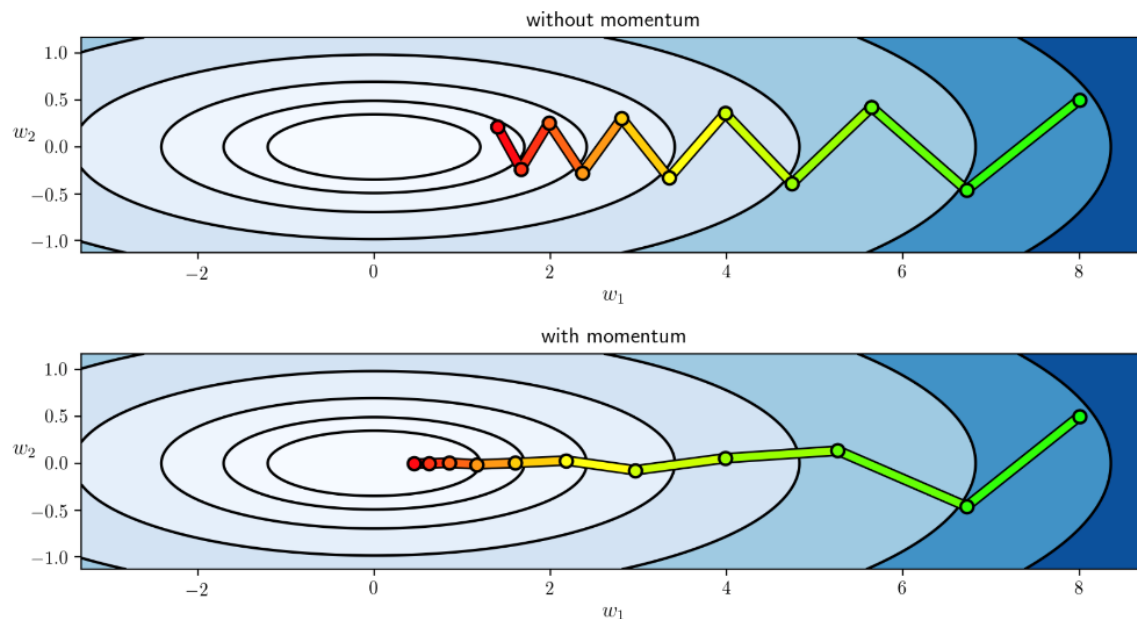
Gradient – Momentum Method

在第 t 次迭代时，计算负梯度的“加权移动平均”作为参数的更新方向，

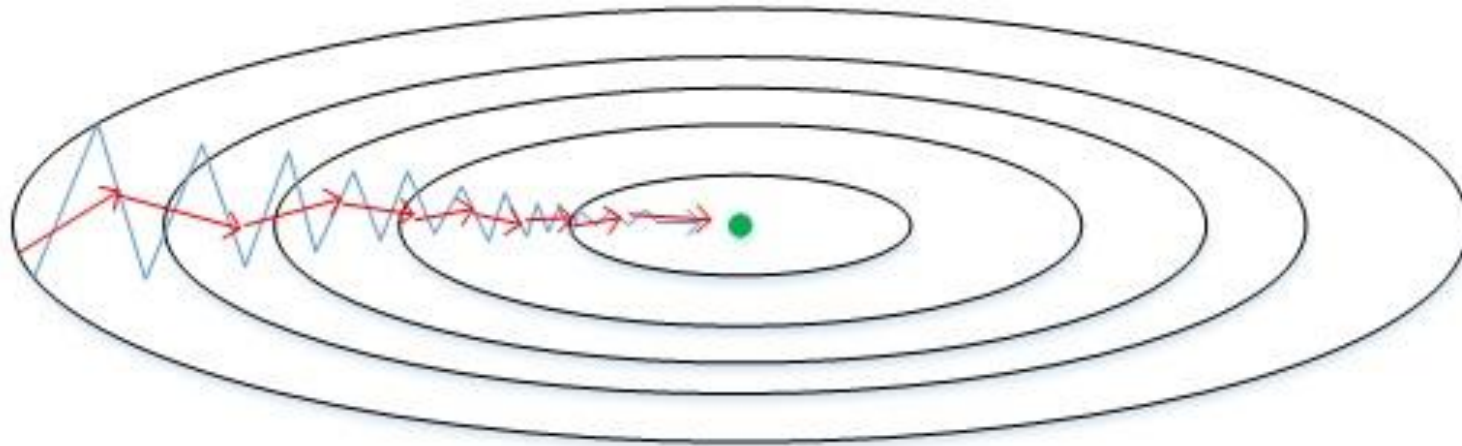
$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t, = -\alpha \sum_{\tau=1}^t \beta^{t-\tau} \mathbf{g}_{\tau}, \quad (7.15)$$

其中 ρ 为动量因子，通常设为 0.9， α 为学习率。

$$\beta^2 \mathbf{v}_{t-2} + \beta \mathbf{g}_{t-1,t-2} + \mathbf{g}_{t,t-1}$$



SGD with momentum

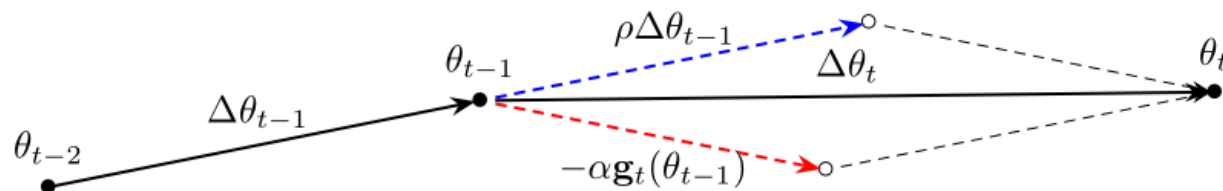


Faster

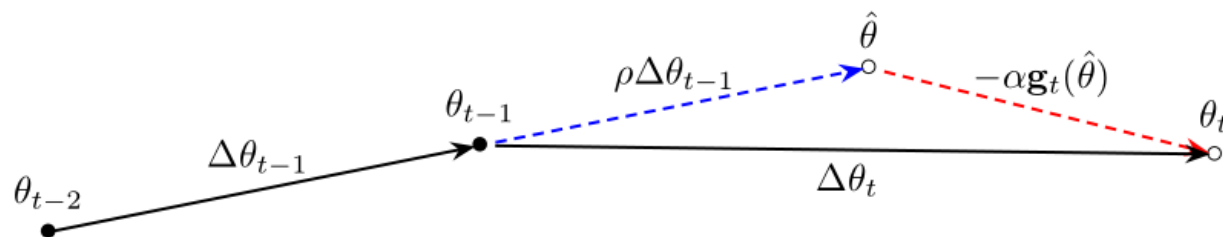
Larger beta, larger momentum

Nesterov accelerate gradient (NAG)

$$\Delta\theta_t = \rho\Delta\theta_{t-1} - \alpha\mathbf{g}_t(\theta_{t-1} + \rho\Delta\theta_{t-1})$$



(a) 动量法



(b) Nesterov 加速梯度

Adam

Momentum + RMSprop

$$M_t = \beta_1 M_{t-1} + (1 - \beta_1) \mathbf{g}_t,$$

$$G_t = \beta_2 G_{t-1} + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t,$$

both the momentum and also the second moment of the gradient

$$\hat{M}_t = \frac{M_t}{1 - \beta_1^t},$$

$$\hat{G}_t = \frac{G_t}{1 - \beta_2^t}.$$

bias correction

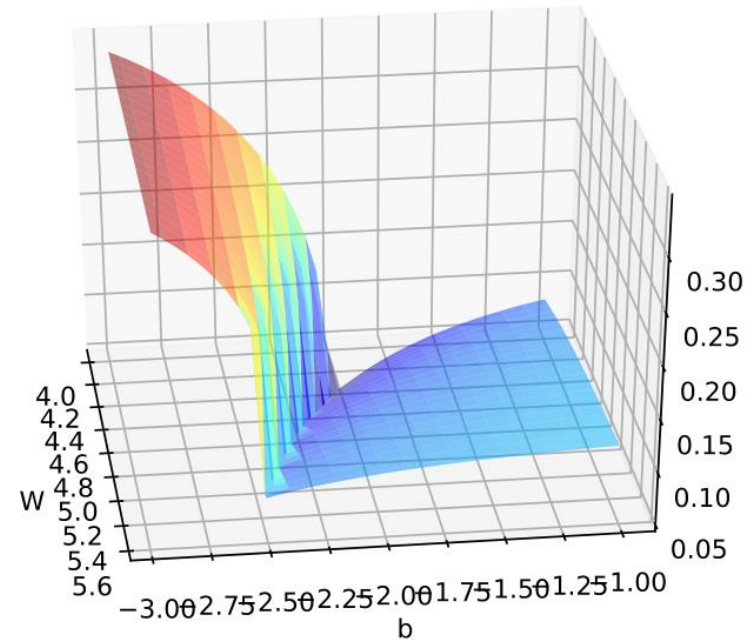
$$\Delta \theta_t = -\frac{\alpha}{\sqrt{\hat{G}_t} + \epsilon} \hat{M}_t$$

RMSprop

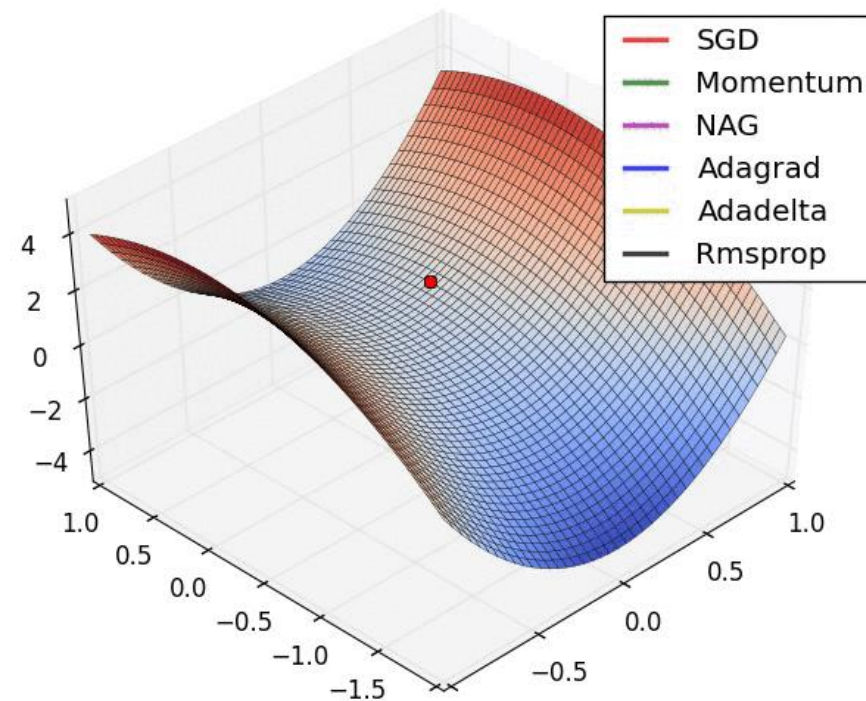
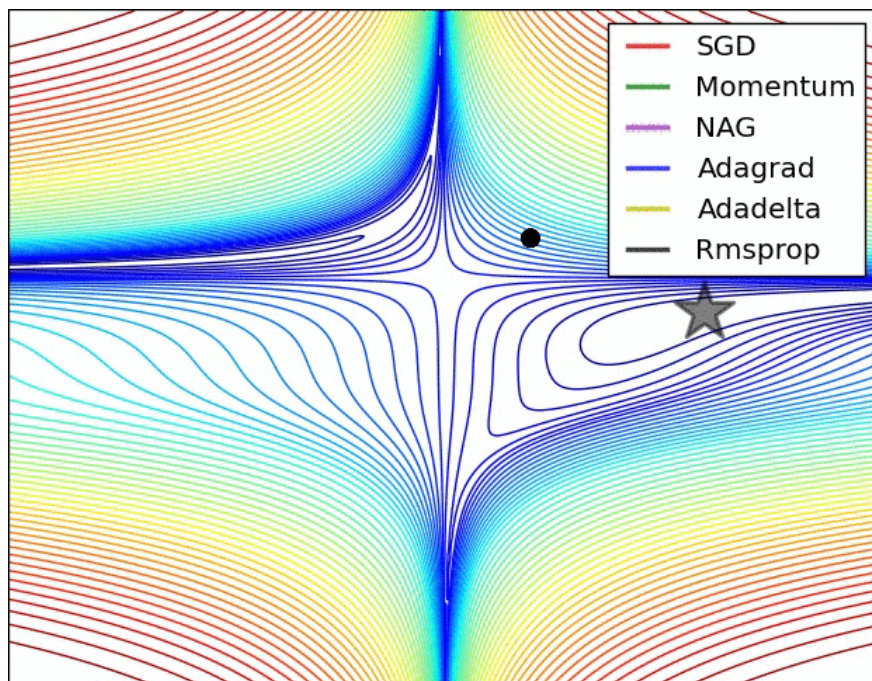
Gradient Clipping

Clipping by value $\mathbf{g}_t = \max(\min(\mathbf{g}_t, b), a)$

Clipping by norm $\mathbf{g}_t = \frac{b}{\|\mathbf{g}_t\|} \mathbf{g}_t$



Optimizers



Summary of optimizer

大部分优化算法可以使用下面公式来统一描述概括：

$$\Delta\theta_t = -\frac{\alpha_t}{\sqrt{G_t + \epsilon}}M_t,$$

$$G_t = \psi(\mathbf{g}_1, \dots, \mathbf{g}_t),$$

$$M_t = \phi(\mathbf{g}_1, \dots, \mathbf{g}_t),$$

$\mathbf{g}_t$ 为第t步的梯度

α_t 为第t步的学习率

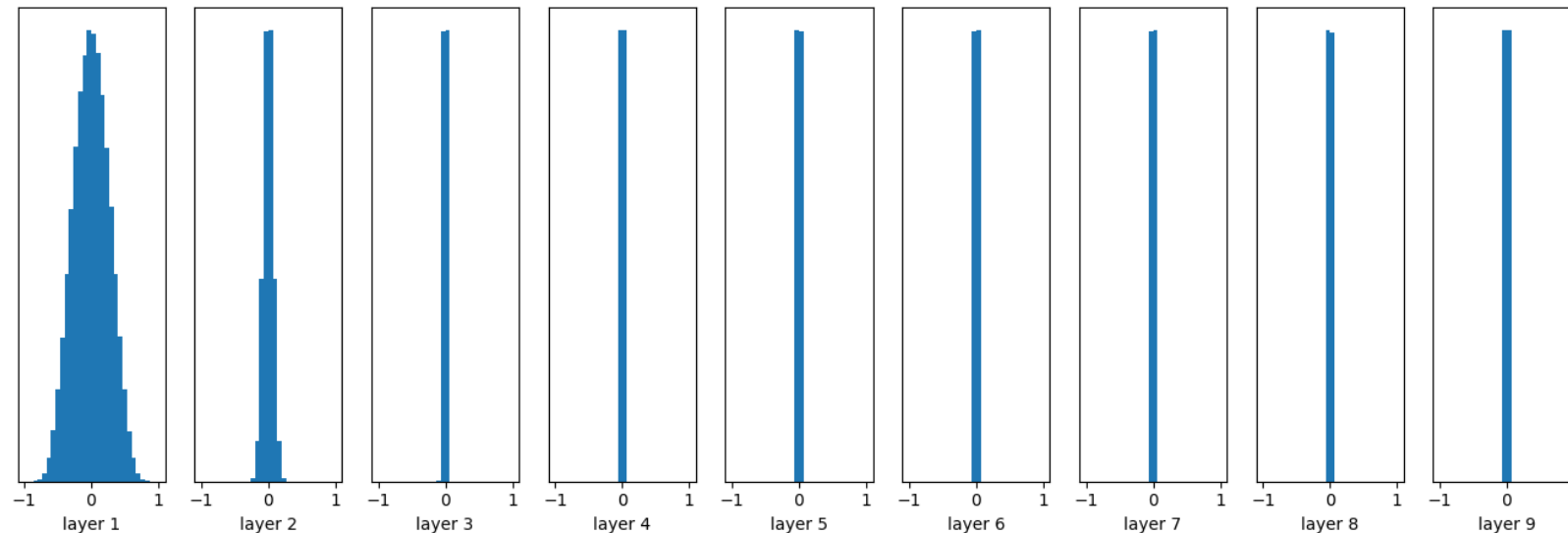
类别		优化算法
学习率调整	固定衰减学习率	分段常数衰减、逆时衰减、(自然)指数衰减、余弦衰减
	周期性学习率	循环学习率、SGDR
	自适应学习率	AdaGrad、RMSprop、AdaDelta
梯度估计修正		动量法、Nesterov 加速梯度、梯度截断
综合方法		Adam $\approx$ 动量法 + RMSprop

Parameter Initialization

- Why not initial parameters as 0?
 - Symmetric weights
- Initialization
 - Pretraining model parameters
 - Random initialization
 - Fix value initialization
 - Bias = 0

Random Initialization

- Gaussian distribution $\mathcal{N}(0, \epsilon^2)$
- Uniform distribution $[-\epsilon, \epsilon]$



Initializations

► 基于方差缩放的参数初始化

► Xavier 初始化和 He 初始化

初始化方法	激活函数	均匀分布 $[-r, r]$	高斯分布 $\mathcal{N}(0, \sigma^2)$
Xavier 初始化	Logistic	$r = 4\sqrt{\frac{6}{M_{l-1}+M_l}}$	$\sigma^2 = 16 \times \frac{2}{M_{l-1}+M_l}$
Xavier 初始化	Tanh	$r = \sqrt{\frac{6}{M_{l-1}+M_l}}$	$\sigma^2 = \frac{2}{M_{l-1}+M_l}$
He 初始化	ReLU	$r = \sqrt{\frac{6}{M_{l-1}}}$	$\sigma^2 = \frac{2}{M_{l-1}}$

► 正交初始化

- 1) 用均值为 0 、 方差为 1 的高斯分布初始化一个矩阵;
- 2) 将这个矩阵用奇异值分解得到两个正交矩阵, 并使用其中之一作为权重矩阵。

Data preprocessing

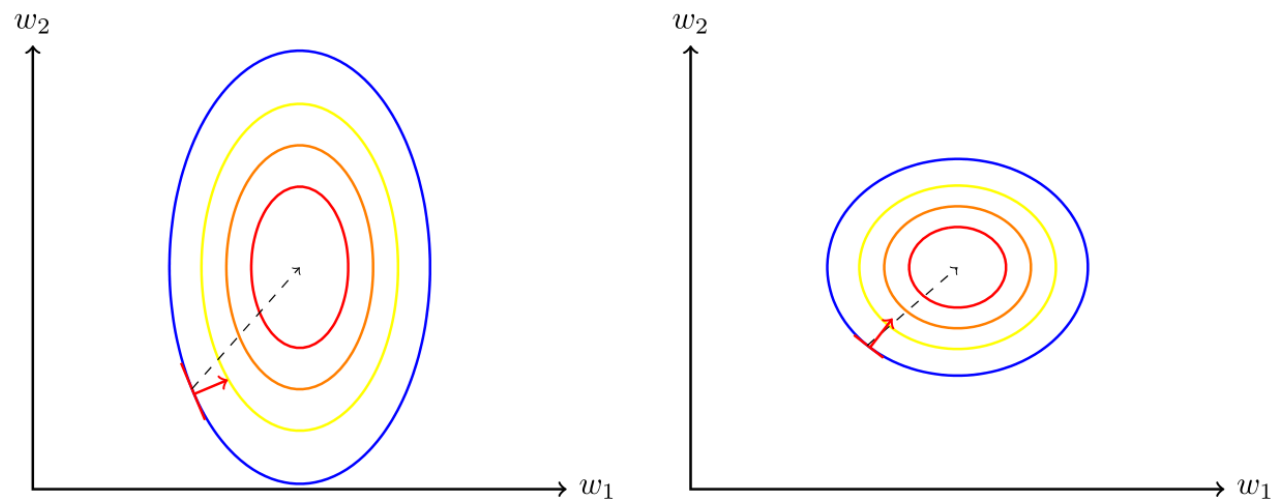
```
# 定义了一个单通道图像预处理的转换管道, 先将图像转换为张量, 然后对图像进行均值和标准差均为0.5的归一化处理  
transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5,), (0.5,))])
```

Effects on Gradient

Data normalization

1. Min-Max normalization
2. Standardization
3. PCA

$$\mu = \frac{1}{N} \sum_{n=1}^N x^{(n)},$$
$$\sigma^2 = \frac{1}{N} \sum_{n=1}^N (x^{(n)} - \mu)^2.$$



(a) 未归一化数据的梯度

(b) 归一化数据的梯度

Review



Next course

- **Regularization**

Chapter 5.5, 5.6 in 《dive into deep learning》

Chapter 9 in 《Deep learning》